

SKILL WEEK – 4

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

```
shreyaj@LAPTOP-FT6VVM4D: ~$ mkdir skill4
shreyaj@LAPTOP-FT6VVM4D: ~$ cd skill4
shreyaj@LAPTOP-FT6VVM4D: ~/skill4$ nano task1.c
shreyaj@LAPTOP-FT6VVM4D: ~/skill4$ gcc task1.c -o task1
./task1
Enter a command: echo Hello World
Argument 1 : echo
Argument 2 : Hello
Argument 3 : World
```

```
GNU nano 8.7.1 task1.c
#include <stdio.h>
#include <string.h>

int main() {
    char input[100];

    printf("Enter a command: ");
    fgets(input, sizeof(input), stdin);

    char *token = strtok(input, " \\t\\n");
    int count = 1;

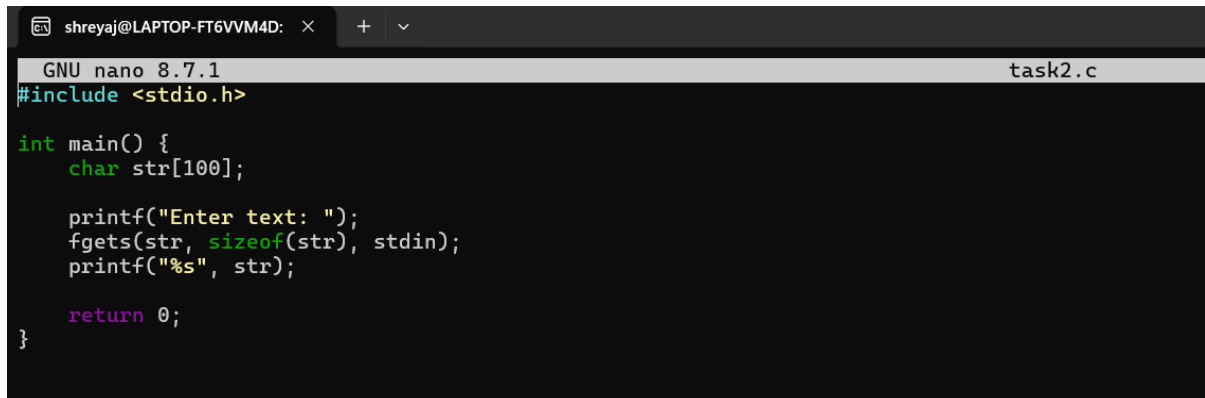
    while (token != NULL) {
        printf("Argument %d : %s\\n", count++, token);
        token = strtok(NULL, " \\t\\n");
    }

    return 0;
}
```

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
shreyaj@LAPTOP-FT6VVM4D:~/skill4$ nano task2.c
shreyaj@LAPTOP-FT6VVM4D:~/skill4$ gcc task2.c -o task2
./task2
Enter text: echo "Hello World"
echo "Hello World"
```



```
shreyaj@LAPTOP-FT6VVM4D: × + v
GNU nano 8.7.1 task2.c
#include <stdio.h>

int main() {
    char str[100];

    printf("Enter text: ");
    fgets(str, sizeof(str), stdin);
    printf("%s", str);

    return 0;
}
```

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```
shreyaj@LAPTOP-FT6VVM4D:~/skill4$ echo "Hello $USER"
Hello shreyaj
```

Task 4: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```
shreyaj@LAPTOP-FT6VVM4D:~/skill4$ mkdir My\ skill4
shreyaj@LAPTOP-FT6VVM4D:~/skill4$ touch task4.txt
shreyaj@LAPTOP-FT6VVM4D:~/skill4$ echo \$HOME
$HOME
```

Task 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```
shreyaj@LAPTOP-FT6VVM4D:~/skill4$ nano task5.c
shreyaj@LAPTOP-FT6VVM4D:~/skill4$ gcc task5.c -o task5
./task5
Parent PID: 1257
Child PID: 1258
'My skill4'   task1.c   task2.c   task5
task1        task2    task4.txt task5.c
Child process completed.
```

```
shreyaj@LAPTOP-FT6VVM4D: × + ▾
GNU nano 8.7.1 task5.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child PID: %d\n", getpid());
        execlp("ls", "ls", NULL);
    } else {
        printf("Parent PID: %d\n", getpid());
        wait(NULL);
        printf("Child process completed.\n");
    }

    return 0;
}
```